

Panduan Penggunaan Google Maps Scraper

Panduan ini menjelaskan cara menyiapkan dan menjalankan scraper Google Maps menggunakan Python virtual environment, Docker Desktop, Selenium, CSV, dan proxy opsional.

1. Prasyarat

Pastikan perangkat sudah memiliki:

- Python 3.10 atau versi lebih baru.
- Docker Desktop yang sudah terpasang dan sedang berjalan.
- Git, jika project belum tersedia di komputer.
- Koneksi internet.

Semua perintah berikut dijalankan dari folder utama project, yaitu folder yang berisi `main.py`, `config.yml`, dan `docker-compose.yml`.

2. Membuat virtual environment

Buka Terminal, PowerShell, atau Command Prompt, lalu masuk ke folder project.

Linux/macOS:

```
python3 -m venv env  
  
source env/bin/activate
```

Windows PowerShell:

```
python -m venv env  
  
.\env\Scripts\Activate.ps1
```

Windows Command Prompt:

```
python -m venv env  
  
.\env\Scripts\activate.bat
```

Jika berhasil, nama `env` biasanya tampil di awal baris terminal.

3. Menginstal requirements

Pastikan virtual environment sudah aktif, lalu jalankan:

```
python -m pip install --upgrade pip

python -m pip install -r requirements.txt
```

Untuk memastikan Selenium dan dependency utama tersedia:

```
python -c "import selenium, yaml, aiohttp; print('Dependency siap')"
```

4. Memastikan Docker Desktop berjalan

Buka Docker Desktop dan tunggu sampai statusnya menunjukkan bahwa Docker Engine sudah running.

Verifikasi dari terminal:

```
docker version

docker compose version
```

Jika perintah tersebut gagal, jalankan Docker Desktop terlebih dahulu.

5. Menjalankan container Selenium

Tanpa proxy

Proxy bersifat opsional. Jika tidak menggunakan proxy, tidak perlu membuat file `.env`.

Jalankan hanya service Selenium:

```
docker compose up -d selenium
```

Periksa statusnya:

```
docker compose ps
```

Service `selenium` harus berada dalam keadaan `Up` atau `running` dan healthcheck-nya harus menjadi `healthy` setelah beberapa saat.

Dengan proxy

Jika proxy sudah dikonfigurasi di file `.env`, jalankan semua service:

```
docker compose up -d --build
```

Periksa status semua service:

```
docker compose ps
```

Selenium tidak bergantung pada `proxy-gateway`. Namun, ketika proxy digunakan, service `proxy-gateway` harus berjalan sehat agar lalu lintas browser dan request detail dapat melewati proxy.

6. Membuka tampilan Selenium

Selenium menyediakan tampilan browser melalui noVNC. Buka alamat berikut di browser:

```
http://localhost:7900
```

Jika halaman meminta password, gunakan:

```
secret
```

Tampilan ini berguna untuk melihat aktivitas Chrome di dalam container. Konfigurasi project menggunakan `Headless: true`, sehingga noVNC dapat digunakan untuk memantau sesi browser tanpa mengubah alur eksekusi scraper.

Endpoint Selenium WebDriver yang digunakan script adalah:

```
http://localhost:4444
```

Jangan membuka port `4444` untuk interaksi manual. Gunakan port `7900` untuk tampilan browser.

7. Menyiapkan file CSV

Buat file CSV, misalnya `queries/queries.csv`.

Kolom wajibnya adalah `query`:

```
query
```

```
coffee shops in Bandung
```

restaurants in Jakarta

hotel di Yogyakarta

Aturan CSV:

- Nama kolom wajib query.
- Satu baris berisi satu pencarian.
- Baris kosong akan diabaikan.
- Spasi di awal dan akhir query akan dihapus.
- Kolom tambahan diperbolehkan, tetapi tidak digunakan.

Contoh dengan kolom tambahan:

query,catatan

coffee shops in Bandung,prioritas

restaurants in Jakarta,normal

Batas jumlah hasil Google Maps

Satu query pencarian Google Maps biasanya hanya menghasilkan maksimal sekitar 120 baris listing. Karena itu, pencarian dengan wilayah yang terlalu luas dapat menghasilkan data yang tidak lengkap. Contohnya, query berikut dapat terpotong:

coffee shop in Bandung

Untuk mendapatkan cakupan yang lebih baik, pecah query berdasarkan wilayah administrasi. Misalnya, alih-alih hanya menggunakan coffee shop in Bandung, gunakan beberapa query berikut:

query

coffee shop di Bandung Wetan

coffee shop di Coblong Bandung

coffee shop di Sukajadi Bandung

coffee shop di Lengkong Bandung

coffee shop di Antapani Bandung

coffee shop di Buahbatu Bandung

Jika hasil di satu kecamatan masih terlalu banyak, pecah lagi berdasarkan nama jalan atau kawasan. Sertakan nama kecamatan dalam query karena nama jalan dapat muncul di lebih dari satu wilayah. Contohnya:

query

coffee shop di Jalan Riau, Kecamatan Bandung Wetan, Bandung

coffee shop di Jalan Dago, Kecamatan Coblong, Bandung

coffee shop di Jalan Braga, Kecamatan Sumur Bandung, Bandung

coffee shop di Jalan Setiabudi, Kecamatan Sukasari, Bandung

coffee shop di Jalan Buah Batu, Kecamatan Buahbatu, Bandung

coffee shop di Jalan Dipatiukur, Kecamatan Coblong, Bandung

Gunakan pola yang sama untuk kategori lain. Contoh untuk restoran di Jakarta:

query

restaurant di Menteng Jakarta Pusat

restaurant di Tebet Jakarta Selatan

restaurant di Kemang Jakarta Selatan

restaurant di Kelapa Gading Jakarta Utara

restaurant di Jalan Senopati Jakarta Selatan

Pemecahan query ini membantu mengurangi risiko batas 120 hasil pada setiap pencarian. Namun, hasil dari query yang berdekatan dapat saling tumpang tindih, sehingga lakukan deduplikasi berdasarkan nama, alamat, URL Google Maps, atau identitas tempat sebelum menggunakan data untuk analisis lebih lanjut.

8. Menjalankan script

Pastikan virtual environment masih aktif dan container Selenium sudah berjalan.

Jalankan script dengan path CSV:

```
python main.py --csvinput queries/queries.csv
```

Contoh menggunakan file sample yang tersedia di project:

```
python main.py --csvinput queries/dummy_queries.csv
```

Script akan:

1. Membuka koneksi ke Selenium pada `http://localhost:4444`.
2. Membuka pencarian Google Maps untuk setiap baris CSV.
3. Mengambil data listing dan detail lokasi.
4. Menyimpan hasil ke SQLite.
5. Mengekspor hasil run ke CSV.

Jika proses dihentikan dengan `Ctrl+C`, hasil yang sudah tersimpan tetap diekspor sebagai hasil parsial.

9. Mengambil file hasil

Secara default, file hasil berada di folder:

```
output/
```

File CSV memiliki format nama:

```
output/YYYYMMDD_HHMMSS_nama_file_csv.csv
```

Database SQLite berada di:

```
output/gmaps_scraper.sqlite
```

Setiap run dibuat sebagai tabel terpisah di database SQLite. Hasil CSV memiliki kolom utama seperti:

- NAME
- LONGITUDE
- LATITUDE
- ADDRESS
- RATING
- RATING_COUNT
- GOOGLE_TAGS
- GOOGLE_URL
- SOURCE_QUERY
- SCRAPED_AT

Buka file CSV menggunakan Excel, LibreOffice Calc, Google Sheets, atau editor teks.

10. Menambahkan proxy

Proxy digunakan secara opsional untuk traffic Chrome dan request detail dari scraper.

Membuat file `.env`

Salin template environment:

Linux/macOS:

```
cp template.env .env
```

Windows PowerShell:

```
Copy-Item template.env .env
```

Buka file `.env`, lalu isi nilai proxy. Contoh:

```
PROXY_HOST=proxy.example.com

PROXY_PORT=8080

PROXY_USER=username_proxy

PROXY_PASSWORD=password_proxy
```

Keterangan:

- `PROXY_HOST`: hostname atau alamat IP proxy.
- `PROXY_PORT`: port proxy.
- `PROXY_USER`: username proxy jika proxy memerlukan autentikasi.
- `PROXY_PASSWORD`: password proxy jika proxy memerlukan autentikasi.

Untuk proxy tanpa autentikasi, `PROXY_USER` dan `PROXY_PASSWORD` dapat dibiarkan kosong:

```
PROXY_HOST=proxy.example.com

PROXY_PORT=8080

PROXY_USER=

PROXY_PASSWORD=
```

Setelah `.env` dibuat, jalankan container:

```
docker compose up -d --build
```

Lalu jalankan scraper seperti biasa:

```
python main.py --csvinput queries/queries.csv
```

Jangan commit atau membagikan file `.env`. File tersebut berisi credential proxy.

Tanpa proxy setelah sebelumnya memakai proxy

Hapus `.env`, atau kosongkan `PROXY_HOST` dan `PROXY_PORT`. Setelah itu jalankan Selenium tanpa proxy:

```
docker compose down
```

```
docker compose up -d selenium
```

```
python main.py --csvinput queries/queries.csv
```

Jika konfigurasi proxy tidak lengkap, script otomatis menonaktifkan proxy dan menjalankan request secara langsung.

11. Menghentikan container

Setelah selesai:

```
docker compose down
```

Perintah ini menghentikan dan menghapus container project, tetapi tidak menghapus file hasil di folder `output/`.

12. Troubleshooting singkat

Selenium tidak dapat diakses

Periksa status container:

```
docker compose ps
```

Lihat log Selenium:

```
docker compose logs selenium
```

Pastikan port 4444 dan 7900 tidak sedang digunakan aplikasi lain.

Tampilan noVNC tidak terbuka

Pastikan Selenium sudah berjalan, lalu buka:

```
http://localhost:7900
```

Gunakan password `secret` jika diminta.

Script melaporkan `Browser_remote_url` bermasalah

Pastikan `config.yml` berisi:

```
Browser_remote_url: http://localhost:4444
```

Pastikan container Selenium berjalan di mesin yang sama dengan script.

Proxy gagal

Periksa hal berikut:

- `PROXY_HOST` dan `PROXY_PORT` tidak kosong.
- Username dan password benar jika proxy memerlukan autentikasi.
- File `.env` berada di folder utama project.
- Container `proxy-gateway` berstatus sehat.
- Tidak ada karakter kutip atau spasi tambahan pada nilai `.env`.

Lihat log proxy:

```
docker compose logs proxy-gateway
```

Jika proxy tidak diperlukan, hapus `.env` dan jalankan hanya service Selenium dengan `docker compose up -d selenium`.